When LLM Labels Fail Together

HanRui Zhang
High School Student, Marc Garneau Collegiate Institute
Toronto, Canada
daniel21cn2016@gmail.com

***Abstract*—Large language models can label many texts in one prompt, reducing repeated instructions and inference cost. This batching also places otherwise separate items in a shared generation, so their errors may no longer be independent. We measure this dependence by matching joint prompts with one-target controls that contain the same neighboring texts in the same order. In the primary batch-16 grid, mean excess error covariance across four open-weight model families and four text-classification tasks was 0.00961 (one-sided randomization $p = 0.0002$). All 16 model–task estimates were positive, and 13 exceeded a prespecified practical threshold. Across four selected model–task combinations, the connection between any two labels weakened as batch size increased, but larger batches created more total dependence, raising the estimated variance from 1.08 to 2.38 times the independent-label value. In a post-execution sensitivity, ignoring which labels came from the same AI call made the results look more certain than they were in three of four model–task combinations. Accounting for calls produced confidence intervals much closer to their intended 95% coverage. An annotation call can therefore act as a statistical cluster. Similar inferential analyses should retain call and protocol provenance and account for call-level dependence.**

***Index Terms*—large language models, data annotation, batch prompting, dependent errors, uncertainty estimation**

I. Introduction

Large language models (LLMs) are increasingly used as text annotators because they can produce useful labels quickly and at low marginal cost. Prior studies have compared LLM labels with those of crowd workers and expert annotators [1], [2]. Broader evaluations show that annotation quality varies across tasks and datasets and still requires human validation [3], [4]; even meaning-preserving prompt-format changes can substantially alter performance [5]. Batch-prompting studies measure efficiency and marginal accuracy [6], [7], while adjacent work establishes adversarial cross-item interference, correlated errors across models, and uncertainty problems under other forms of prompt or interaction dependence [8], [9], [10], [11]. We connect these lines by estimating joint-minus-exact-context annotation-error covariance and measuring its coverage consequences under repeated sampling.

Marginal accuracy does not reveal whether errors are independent. When 16 texts share one prompt and one generated response, the model can carry a local interpretation or formatting decision across the entire call. If errors become positively correlated within calls, a dataset with n labels contains less independent information than its row count suggests. Confidence intervals that treat the labels as independent may then be too narrow even when their point estimates are unbiased. Classical clustered-data methods address this same gap between row count and independent information [12], [13]. This is a design issue rather than a property visible from accuracy alone.

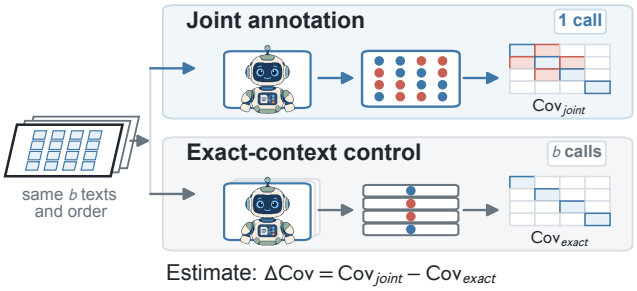


Fig. 1. Matched-context design for a batch of b texts. The joint condition returns b labels in one call, whereas the control generates each target label in one of b separate calls using the same visible context. Covariance matrices are schematic; the estimand subtracts exact-context covariance from joint-call covariance. The cross-setting grid and recovered coverage sensitivity use $b = 16$; the batch-size study uses $b \in \{2, 4, 8, 16, 32\}$ in four prespecified cells.

We test whether the joint annotation-call treatment creates across-item error covariance (Fig. 1). Each joint prompt is paired with an exact-context control that contains the same documents in the same order but requests only one target label. This holds the visible context fixed while changing the full output request from joint multi-target generation to separate target-only generations. The study comprises a cross-setting grid covering four model families and four tasks at batch size 16, a batch-size analysis at sizes 2, 4, 8, 16, and 32 in one prespecified cell per family, and a recovered post-execution sensitivity at batch size 16 comparing ordinary item-level intervals with call-aware intervals.

The study makes two contributions. First, it identifies recurrent excess error covariance across a balanced 4×4 model–task grid at batch size 16 and, in one cell per family, separates decreasing pairwise dependence from increasing cluster-level information loss across sizes 2–32. Second, a recovered repeated-sampling sensitivity estimates the coverage loss caused by ignoring annotation-call membership. The practical recommendation is to store a call identifier with every LLM-generated label and use that identifier when estimating uncertainty.

II. Related Work

LLMs are increasingly evaluated as substitutes or complements for human annotators. Gilardi et al. found that ChatGPT could outperform crowd workers on several text-annotation tasks, and Tan et al. organize the broader literature around annotation generation, assessment, and downstream use [1], [2]. Wider evaluations qualify this promise. Ziems et al. found substantial variation across models and computational social science tasks, while Pangakis and Wolken found that performance varied across 27 tasks drawn from 11 protected datasets and argued for validation against human labels [3], [4]. Seemingly innocuous prompt-format choices can also change model performance [5]. These studies establish that LLM annotation quality depends on the task and protocol, but their main outcomes are item-level accuracy, agreement, or sensitivity. Kim et al. instead measure error correlation across distinct models [9]; our estimand concerns dependence among items labeled within one shared generation.

Batch prompting is the closest computational setting to our study. Cheng et al. evaluate its token, time, and accuracy tradeoffs across ten datasets [6]. Pipal et al. study annotation more directly across eight production models, four tasks, and batch sizes up to 1,000 [7]. Both focus on efficiency and marginal coding performance rather than covariance among labels sharing a response. Lin et al. show that ordinary batch-prompt performance depends on item position and order because earlier outputs change the decoder context [14]; Yue and Yao show that a malicious instruction in one item can disrupt other outputs in the same batch [8]. We instead estimate error covariance under ordinary classification inputs against target-only calls with the same visible context.

Dependence-aware inference is also established outside batched annotation. Gligorić et al. use LLM confidence and selective human annotation to construct valid intervals [15]. Llewellyn et al. cluster semantically dependent benchmark prompts, while Zeng et al. address missing responses under user-level dependence in human–AI interactions [10], [11]. These studies address annotation error, prompt structure, or users; we test dependence created when multiple labels share one generation. Standard methods already accommodate known clusters [12], [13], [16], [17]. Our contribution is not a new estimator: it is a matched measurement of call-associated annotation-error covariance and a repeated-sampling evaluation of the coverage loss from discarding call membership.

III. Experimental Design

A. Models and Tasks

We evaluated Qwen3-8B, Granite-3.3-8B-Instruct, Llama-3.1-8B-Instruct, and Mistral-7B-Instruct-v0.3. These similarly sized open-weight instruction models represent four model families [18], [19], [20], [21]. The tasks were toxicity classification on Civil Comments [22], natural-language inference on MultiNLI [23], sentiment classification on IMDb [24], and topic classification on AG News [25]. For Civil Comments, validation-set human toxicity annotation fractions were binarized as toxic at values ≥ 0.5. After fixed length and prior-study exclusions, candidates were placed in a deterministic hash order, and the fixed confirmation segment was divided into 16 equal length strata. Dataset labels were treated as reference labels, so all reported errors are reference-relative rather than claims of adjudicated semantic truth.

Every model used the same instruction template, JSON response grammar, parser, greedy decoding, and 32,768-token context limit. The experiment used 8,192 fixed items per task. This item count was fixed before any joint outputs were observed using solo-prompt error rates and CPU power simulations. No cell was removed based on model accuracy or parse rate.

B. Joint and Exact-Context Conditions

For each model–task cell, we formed length-balanced groups of 16 items under three independent randomized assignments. In the *joint* condition, one prompt requested labels for all 16 items. In the *exact-context* condition, the prompt showed those same 16 items in the same order but requested the label of only one target; repeating this for all targets produced one label per item in separate calls. This control holds the visible context constant while changing whether labels share a generation.

Let $e_i \in \{0, 1\}$ indicate disagreement with the reference label. For each model–task–condition–assignment, $\bar{e}$ is the mean error across all item rows in that condition. For both conditions, fixed item errors were regrouped by the matched assigned 16-item group: one joint call or the corresponding 16 separate exact-context calls. Within each group, we averaged $(e_i - \bar{e})(e_j - \bar{e})$ over distinct item pairs, then averaged the three assignment-specific contrasts. The primary cell statistic was

$$\Delta = \mathrm{Cov}_{\mathrm{joint}}(e_i, e_j) - \mathrm{Cov}_{\mathrm{exact}}(e_i, e_j). \tag{1}$$

A positive Δ means that errors co-occur more often when labels share a response than when they merely share context. Under a sharp no-assignment-effect null holding outputs fixed, 4,999 paired replays permuted items within assignment, length stratum, and execution block while preserving group size and repeated appearances; both conditions used the same regrouping. We used upper-tail p-values and adjusted the 16 cell tests by Holm's method. The shift interval subtracts mean-centered replay quantiles and is conditional on the fixed outputs and replay design. Before observing joint results, we defined $\Delta \geq 0.002$ as a practically relevant point-estimate floor. Replay rejection and whether a point estimate met this floor were evaluated separately. The equal-cell panel estimate gives equal weight to every model–task cell, conditional on the fixed models, tasks, and item samples.

Malformed outputs were retained under prespecified treatments that counted unresolved targets as correct, counted them as errors, dropped only unresolved targets, or dropped their whole calls. This prevents selective parsing from creating the result.

C. Batch-Size Robustness

We repeated the joint-versus-exact comparison in one prespecified cell per model family at batch sizes 2, 4, 8, 16, and 32. Every size used all 8,192 items under three nested randomized assignments and a uniform structured-output contract. The primary contrast standardized each covariance by its condition-specific Bernoulli variance, yielding excess within-call error correlation. For each cell, we regressed this contrast on $\log_2$ batch size; the summary statistic was the equal-cell mean slope. Because the same items recur across assignments, we report this analysis descriptively with the slope from each complete assignment. The artifact retains the prespecified size-32 block bootstrap as a sensitivity, but its separately resampled assignments do not preserve recurring item identity and are not used for headline uncertainty. We transformed pooled excess correlation ρ_Δ into the descriptive excess design effect $1 + (b - 1)\rho_\Delta$ and variance-equivalent effective sample size $8{,}192/[1 + (b - 1)\rho_\Delta]$.

D. Recovered Repeated-Sampling Sensitivity

The repeated-sampling component tested whether call dependence changes uncertainty for mean reference-relative error. Granite/IMDb, Qwen/Civil Comments, Llama/Civil Comments, and Mistral/MultiNLI were each family's first admitted cell in a frozen task order, selected from solo evidence before covariance outputs. Each of 250 reference replicates and 400 evaluation replicates sampled 512 item appearances and generated 32 fresh 16-item calls. With 400 evaluation replicates, the maximum binomial Monte Carlo standard error of a coverage estimate is 2.5 percentage points. The reference replicates estimated the target error rate. For each evaluation replicate, we constructed three nominal 95% intervals: an item-level Wald interval treating 512 errors as independent, a t interval over the 32 call means, and a whole-call bootstrap interval. Frozen reference-precision and target-shift checks supported undercoverage for Granite, Qwen, and Mistral.

A technical recovery completed 1,095 of 83,200 calls (1.3%) that initially produced malformed or truncated responses. The reruns retained the original input prompt, model, seed, precision, runtime, and decoding settings; only the enforced output schema and maximum response allowance changed. All originally successful outputs were retained. The original coverage result was unavailable, so the recovered coverage analysis is reported as a post-execution sensitivity. The untouched covariance analysis remains primary and includes all prespecified malformed-output bounds.

E. Artifact Availability

The reviewer artifact contains analysis code, frozen protocols and prompts, dataset indices, and analysis-ready error arrays. It reproduces the reported analyses but cannot audit model execution or parsing because dataset text, raw model prose, and cloud execution records are excluded. It is available at https://anonymous.4open.science/r/batched-annotation-D206/README.md.

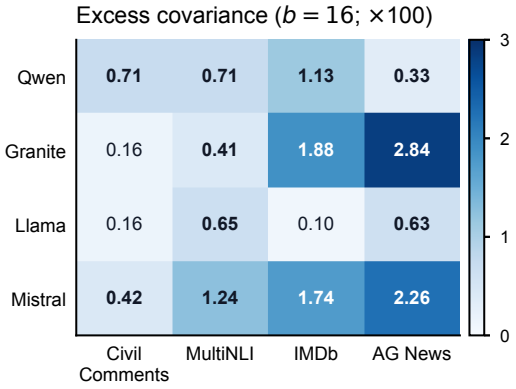


Fig. 2. Joint-minus-exact error covariance at batch size 16 (×100; 8,192 items per cell). Bold point estimates meet the prespecified 0.002 practical floor; all 16 cells reject the sharp null after Holm adjustment.

IV. Results

A. Dependence Recurs Across Models and Tasks

The equal-cell excess covariance was 0.00961, with a 95% design-respecting shift interval of $[0.00936, 0.00986]$ and one-sided replay $p = 0.0002$. All 16 cells rejected the sharp null after Holm adjustment; 13 point estimates met the separate 0.002 practical floor. Those 13 cells covered all four model families and all four tasks (Fig. 2). Every leave-one-model-out and leave-one-task-out mean remained positive, so the panel aggregate was not driven by a single family or dataset.

Across the four prespecified malformed-output rules, panel-aggregate excess covariance ranged from 0.00752 to 0.01550, always above the practical threshold. A sensitivity substituting 3,002 recovered calls gave 0.00790; 13 cells met its historical effect-plus-replay-rejection rule, with two cells exchanging membership. All 48 assignment contrasts were positive, and leave-one-assignment means ranged from 0.00935 to 0.00992. Across cells and assignments, standardized covariance averaged 0.0557 joint and 0.0014 exact-context, giving mean design effects of 1.835 and 1.021. Separately averaged effective sample sizes were 5,022 and 8,105 per 8,192 labels.

A post-hoc Civil Comments sensitivity kept four-cell covariance positive at thresholds 0.4 (0.00343), 0.5 (0.00464), and 0.6 (0.00626), and after excluding 509 ambiguous items (0.00524). Both Civil Comments batch-size slopes remained negative, and the repeated-sampling conclusions were qualitatively unchanged.

The information loss was heterogeneous across settings. A descriptive exchangeable-correlation calculation gave cell-level excess design effects from 1.14 to 2.87, corresponding to effective sample sizes from 7,162 down to 2,850 for 8,192 labeled rows. As a negative control, fake batches assembled from separately generated solo labels had covariance between −0.00086 and 0.00082 across the 16 cells. Regrouping independent labels therefore did not reproduce the dependence observed when labels shared a response.

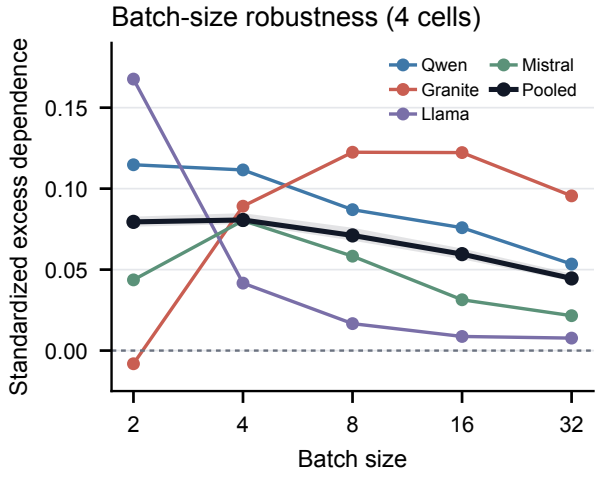


Fig. 3. Standardized joint-minus-exact dependence across batch sizes 2–32 in four prespecified cells. The black line is the equal-cell mean; shading spans the three complete randomized assignments.

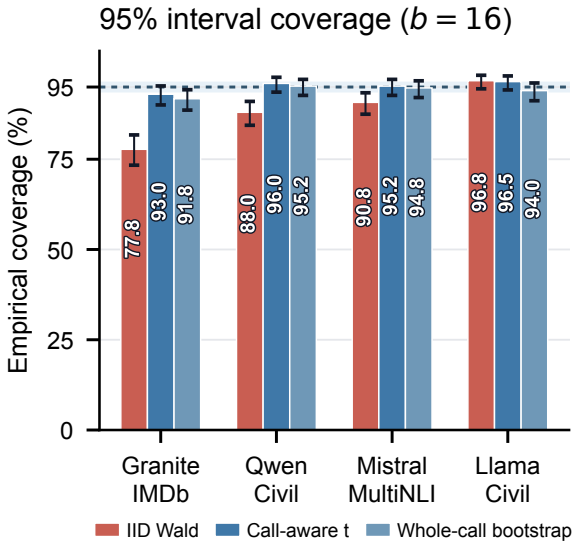


Fig. 4. Recovered post-execution sensitivity of empirical coverage across 400 replicates at batch size 16. Error bars are exact 95% binomial intervals; shading marks 94–96% coverage.

B. Dependence Persists but Does Not Grow Uniformly with Batch Size

Pooled standardized excess dependence was positive at every tested size, measuring 0.079 at batch size 2, 0.081 at size 4, and 0.045 at size 32 (Fig. 3). Its equal-cell slope was −0.00910 per doubling. The pooled slope was negative in each complete assignment (−0.00960, −0.00879, and −0.00889). Pairwise excess dependence therefore decreased on average after size 4, although this three-assignment comparison is descriptive.

Three cells had negative slopes, whereas Granite/IMDb had a positive slope; no single trend described every selected cell.

Pairwise decline did not prevent the cluster-level penalty from growing. The descriptive excess design effect increased monotonically from 1.079 at batch size 2 (assignment range 1.076–1.083) to 2.381 at size 32 (2.328–2.473). The corresponding variance-equivalent sample size relative to the exact-context baseline fell from 7,589 to 3,440 out of 8,192 rows. Each pair became less dependent on average, but every label shared its call with more labels.

C. Recovered Sensitivity: Ignoring Calls Can Understate Uncertainty

The recovered repeated-sampling sensitivity connects covariance to a familiar inferential quantity. Figure 4 reports coverage with exact Monte Carlo intervals. For Granite/IMDb, item-level coverage was 77.8% and empirical variance was 2.71 times its nominal value. The call-aware t interval widened appropriately and reached 93.0% coverage. Qwen/Civil Comments showed the same pattern, with coverage increasing from 88.0% to 96.0%.

Mistral/MultiNLI had milder undercoverage (90.8%) and improved to 95.3% with the call-aware interval. Llama/Civil Comments showed no item-level undercoverage. Whole-call bootstrap coverage was 91.8%, 95.3%, 94.8%, and 94.0% for Granite, Qwen, Mistral, and Llama, respectively. Wilson-score intervals reproduced the item-level result, so undercoverage was not a Wald-boundary artifact. The finding is therefore not that every batched annotation dataset has invalid intervals. Rather, positive dependence creates a risk that cannot be diagnosed from the number of labeled rows alone.

V. Discussion

The unit that generates an annotation can also be a unit of statistical dependence. A joint prompt may establish a temporary decision rule, interpretation, or output pattern that affects several labels at once. Exact-context controls show that the observed covariance is not explained solely by longer prompts or neighboring documents. We therefore treat the joint call as an analysis cluster, not as an identified mechanism. Exploratory analysis found no clear linear trend in the marginal error contrast by output position, while pair covariance was stronger for nearby positions; this pattern does not identify a mechanism.

The batch-size result separates pairwise dependence from cluster-level information loss. Dependence remained detectable even in small batches, while three selected cells decreased and one increased across the tested range. Nevertheless, the implied excess design effect rose with batch size because the number of other labels sharing each call grew faster than pairwise dependence declined. Reducing a batch can reduce this penalty, but it does not justify treating the remaining labels as independent.

This distinction matters for mean- or proportion-like estimates under similar designs. Positive within-call covariance can make row-independent formulas underestimate variance.

Retaining original and parent call IDs, attempt and recovery status, model and protocol versions, decoding settings, batch size, and output position permits call-aware analysis and auditing. Call-level t intervals, cluster-robust standard errors, or whole-call bootstrap resampling can address call clustering [13], [16], [17]; the choice depends on the number and structure of independent calls, and few-call settings require small-sample procedures. Crossed source, user, conversation, or document clusters require methods that account for those dimensions too. These methods change uncertainty, not label accuracy. Effects on other downstream estimands remain untested.

The study has four limitations. First, it evaluates four open-weight models near the 7–8B scale and four English classification tasks; proprietary, larger, multilingual, generative, and subgroup-specific behavior may differ. Second, the batch-size experiment uses four different model–task cells and sizes 2–32, so it does not separate model-family effects from task effects or establish a universal scaling law. Third, the matched control identifies the bundled effect of joint annotation, not whether multi-target instruction, shared-response decoding, or another component produces the dependence. Fourth, the repeated-sampling study required post-execution recovery of 1,095 calls under a modified output schema and response allowance. Repair-free IID coverage was directionally similar for Granite/IMDb (78.9%; 105/133 replicates) and Qwen/Civil Comments (88.4%), versus 77.8% and 88.0% after recovery, but these subsets were too small for the prespecified decision rule. Coverage therefore remains sensitivity evidence.

VI. Conclusion

The primary grid showed excess error covariance across four model families and four tasks at batch size 16. In the descriptive four-cell batch-size analysis, pairwise dependence decreased on average, but the implied design effect increased. In the recovered sensitivity, treating 512 labels as independent produced substantial undercoverage in two cells and milder undercoverage in a third, while intervals based on 32 call means were much closer to 95%. Similar analyses should retain call and protocol provenance, report dependence separately from accuracy, and account for every grouping in the design.

References

[1] F. Gilardi, M. Alizadeh, and M. Kubli, "ChatGPT outperforms crowd workers for text-annotation tasks," *Proceedings of the National Academy of Sciences*, vol. 120, no. 30, p. e2305016120, 2023.

[2] Z. Tan, D. Li, S. Wang, A. Beigi, B. Jiang, A. Bhattacharjee, M. Karami, J. Li, L. Cheng, and H. Liu, "Large language models for data annotation and synthesis: A survey," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 930–957.

[3] C. Ziems, W. Held, O. Shaikh, J. Chen, Z. Zhang, and D. Yang, "Can large language models transform computational social science?" *Computational Linguistics*, vol. 50, no. 1, pp. 237–291, 2024.

[4] N. Pangakis and S. Wolken, "Keeping humans in the loop: Human-centered automated annotation with generative AI," in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 19, no. 1, 2025, pp. 1471–1492.

[5] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting," in *International Conference on Learning Representations*, 2024.

[6] Z. Cheng, J. Kasai, and T. Yu, "Batch prompting: Efficient inference with large language model APIs," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2023, pp. 792–810.

[7] C. Pipal, E.-M. Vogel, M. Wack, and F. Esser, "Researchers waste 80% of LLM annotation costs by classifying one text at a time," *arXiv preprint arXiv:2604.03684*, 2026.

[8] M. Yue and Z. Yao, "Efficient but vulnerable: Benchmarking and defending LLM batch prompting attack," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 4746–4761.

[9] E. M. Kim, A. Garg, K. Peng, and N. Garg, "Correlated errors in large language models," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 30 038–30 066.

[10] M. Llewellyn, I. Thornton, J. Bishop, and A. Gray, "Correcting prompt dependence in LLM benchmarks: A Bayesian hierarchical model with embedding-space clustering," *arXiv preprint arXiv:2510.05709*, 2025.

[11] Z. Zeng, D. Arbour, A. Feller, I. Dasgupta, A. R. Sinha, and E. H. Kennedy, "Handling missing responses under cluster dependence with applications to language model evaluation," in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 56 527–56 556.

[12] K.-Y. Liang and S. L. Zeger, "Longitudinal data analysis using generalized linear models," *Biometrika*, vol. 73, no. 1, pp. 13–22, 1986.

[13] A. C. Cameron and D. L. Miller, "A practitioner's guide to cluster-robust inference," *Journal of Human Resources*, vol. 50, no. 2, pp. 317–372, 2015.

[14] J. Lin, M. Diesendruck, L. Du, and R. Abraham, "BatchPrompt: Accomplish more with less," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=Agyicd577r

[15] K. Gligorić, T. Zrnic, C. Lee, E. Candès, and D. Jurafsky, "Can unconfident LLM annotations be used for confident conclusions?" in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2025, pp. 3514–3533.

[16] J. G. MacKinnon, M. Ø. Nielsen, and M. D. Webb, "Cluster-robust inference: A guide to empirical practice," *Journal of Econometrics*, vol. 232, no. 2, pp. 272–299, 2023.

[17] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 69, no. 3, pp. 369–390, 2007.

[18] A. Yang, A. Li, B. Yang *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[19] IBM Granite Team, "Granite-3.3-8B-Instruct model card," Hugging Face, 2025. [Online]. Available: https://huggingface.co/ibm-granite/granite-3.3-8b-instruct

[20] A. Dubey, A. Jauhri, A. Pandey *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[21] Mistral AI, "Mistral-7B-Instruct-v0.3 model card," Hugging Face, 2024. [Online]. Available: https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3

[22] D. Borkan, L. Dixon, J. Sorensen, N. Thain, and L. Vasserman, "Nuanced metrics for measuring unintended bias with real data for text classification," in *Companion Proceedings of the 2019 World Wide Web Conference*, 2019, pp. 491–500.

[23] A. Williams, N. Nangia, and S. R. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2018, pp. 1112–1122.

[24] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 2011, pp. 142–150.

[25] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Advances in Neural Information Processing Systems 28*, 2015, pp. 649–657.